

CS795/895: Generative AI — Project Milestone Report

Title: From Vertex AI to TinyLLM: Building Edge-Ready AI for Personalized Healthcare

Team Members: Anton Rasmussen

1. Materials and Methods

This project investigates how to compress, quantize, and deploy foundation models for edge-compatible, privacy-preserving inference. The workflow bridges Google Vertex AI model deployment with local TinyLLM builds that can run on CPU or embedded systems via llama.cpp.

Data & Tools

Component	Details
Cloud Platform	Google Cloud Vertex AI
Dataset	Iris Dataset (via sklearn.datasets.load_iris)
Modeling Frameworks	scikit-learn, PyTorch, Hugging Face Transformers
Quantization Libraries	GPTQ, AWQ (Applied to Phi-3 Mini / Mistral-7B models)
Local Runtime	llama.cpp GGUF builds for CPU inference
Development Environment	Google Colab & Vertex AI Workbench (Python 3.11), VSCode for local testing

Step-by-Step Method

1. Trained a RandomForestClassifier on the Iris dataset and exported model using joblib for Vertex AI
2. Deployed model via vertex_ai.Model.upload() and verified endpoint predictions
3. Created JSONL batch input and executed batch prediction with specified machine type

4. Applied GPTQ and AWQ quantization to Phi-3 Mini and Mistral-7B models [In Progress]

5. Exported models to GGUF format and verified local CPU inference via llama.cpp [To do]

Rationale: Vertex AI provides scalable deployment while TinyLLM explores decentralized, secure inference for healthcare applications where sensitive data must remain local.

2. Goals & Task Plan

Goal / Task	Status	Notes
Finalize Vertex AI setup; deploy Iris model	✅ Completed	Confirmed working deployment
Run and debug batch prediction jobs	✅✅ Partial	Resolved machine type error
Quantize Phi-3 Mini	✅ In Progress	To apply GPTQ quantization
Configure llama.cpp and test CPU inference	✅ To Do	To successfully load quantized model
Document methods and prepare report	✅ In Progress	Completed Poster draft

Risks & Mitigations:

- Compute limits on Vertex AI: Used smaller models.
- LLM file size: Leveraged 4-bit quantization.
- Batch job errors: Added explicit machine type and verified GCS paths.

3. Results and Outcomes

Vertex AI endpoint successfully responded to prediction requests from JSONL input.

Example prediction response:

```
{'predictions': ['setosa', 'versicolor', 'virginica']}
```

4. Code and Running Results

See: [Colab Project](#)

Key Files:

- `quantize_gptq.py` – Applies GPTQ quantization to Phi-3 Mini; exports quantized weights
- `quantize_awq.py` – Applies AWQ (activation-aware) quantization; exports quantized weights
- `evaluate.py` – Evaluates the quantized TinyLLM (e.g., Phi-3 Mini or Mistral-7B) on held-out prompts; logs accuracy, latency, and token throughput
- `export_to_gguf.py` – Converts the quantized model to GGUF format compatible with `llama.cpp` for CPU inference
- **[TBD]** `run_llamacpp_cpu.sh` – Runs `llama.cpp` with the exported GGUF

5. Roles and Contributions (Sep 30 – Oct 20)

Team Member	Role	Contribution
Anton Rasmussen	Lead Researcher	Vertex AI deployment, debugging batch prediction pipeline, beginning GPTQ/AWQ quantization, documentation and report authoring.

6. Next Steps

- Complete GPTQ/AWQ quantization and `llama.cpp` testing
- Evaluate accuracy, latency, and memory footprint across CPUs
- Explore collaborative inference across edge devices
- Prepare final poster and demonstration and complete final report